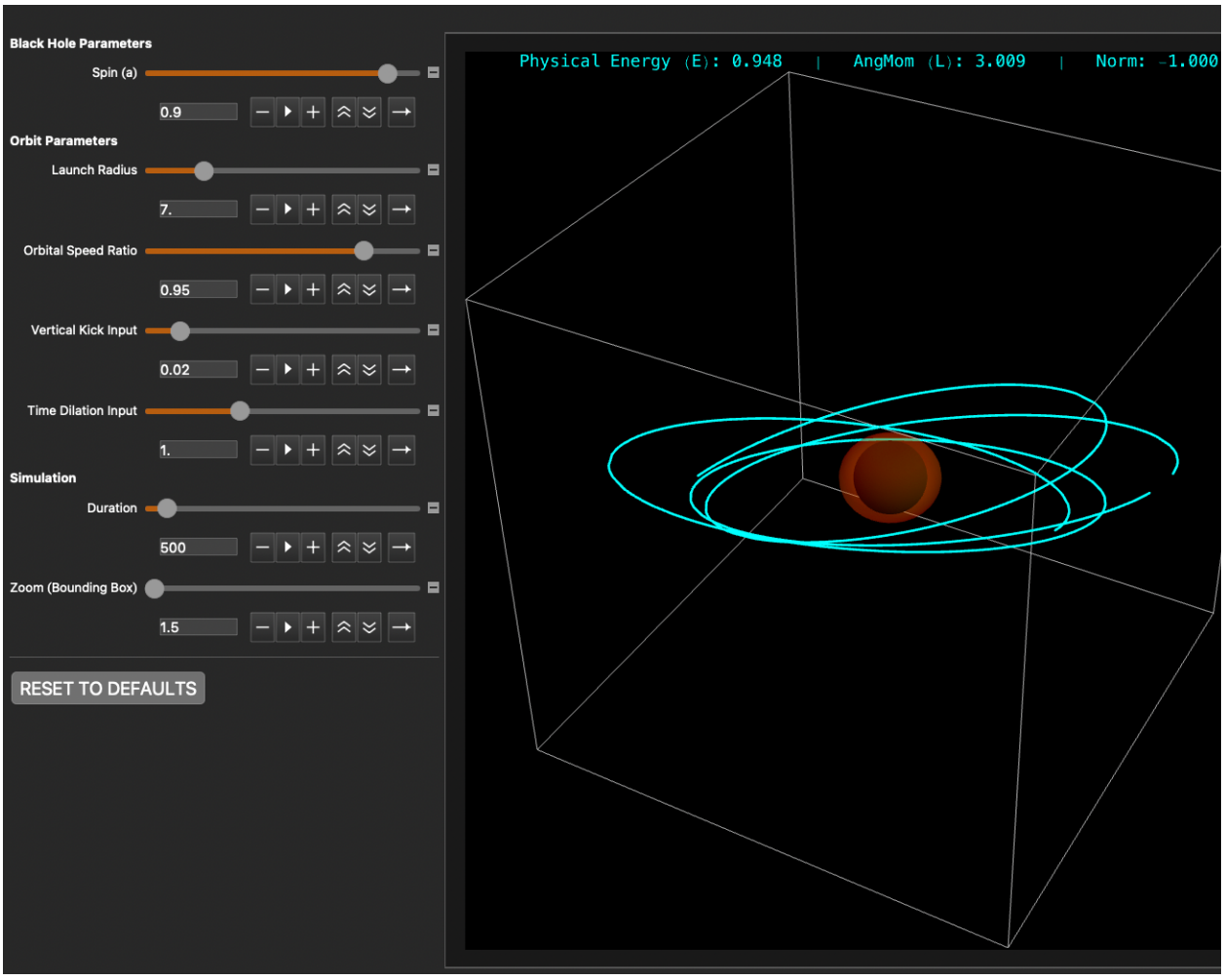
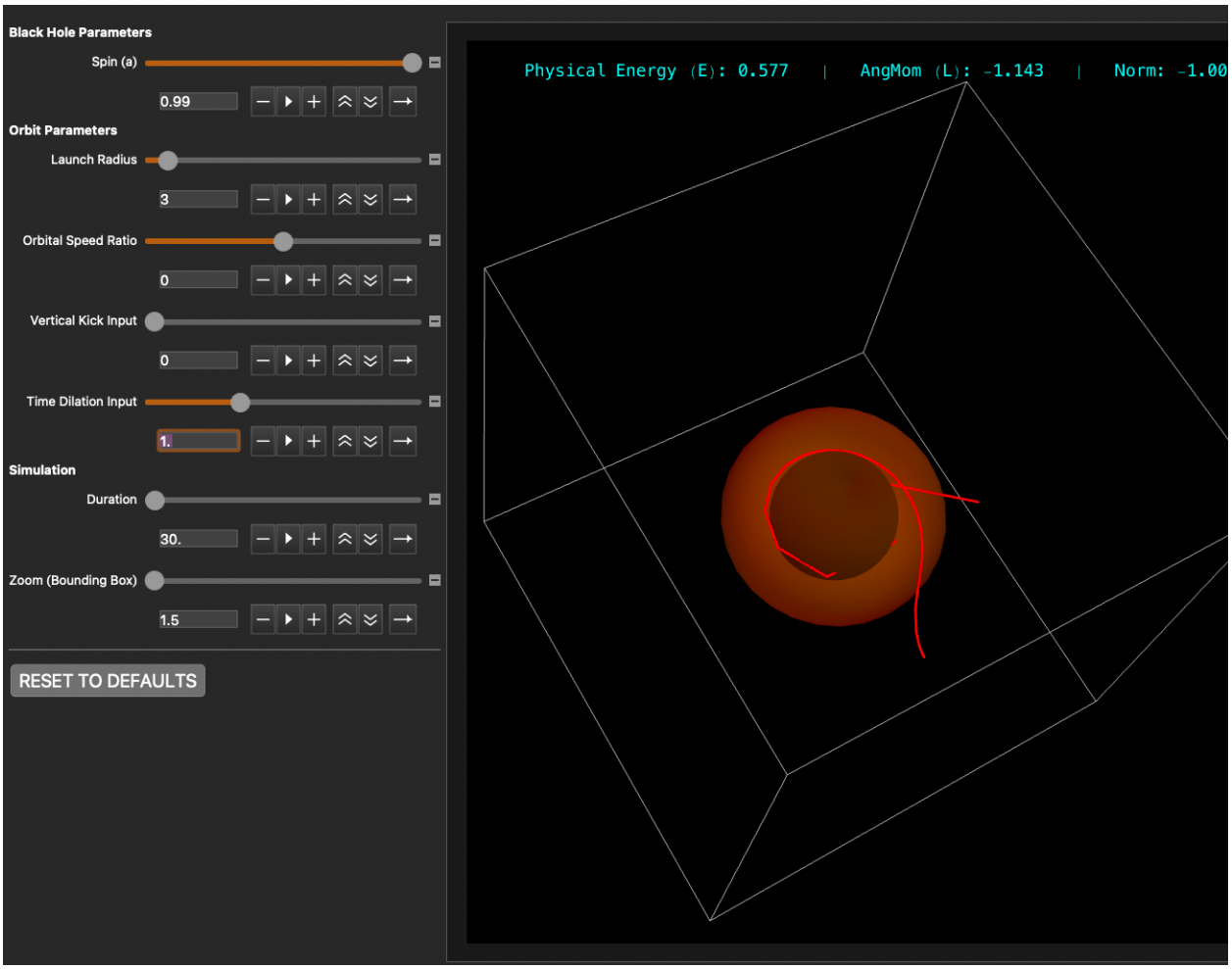
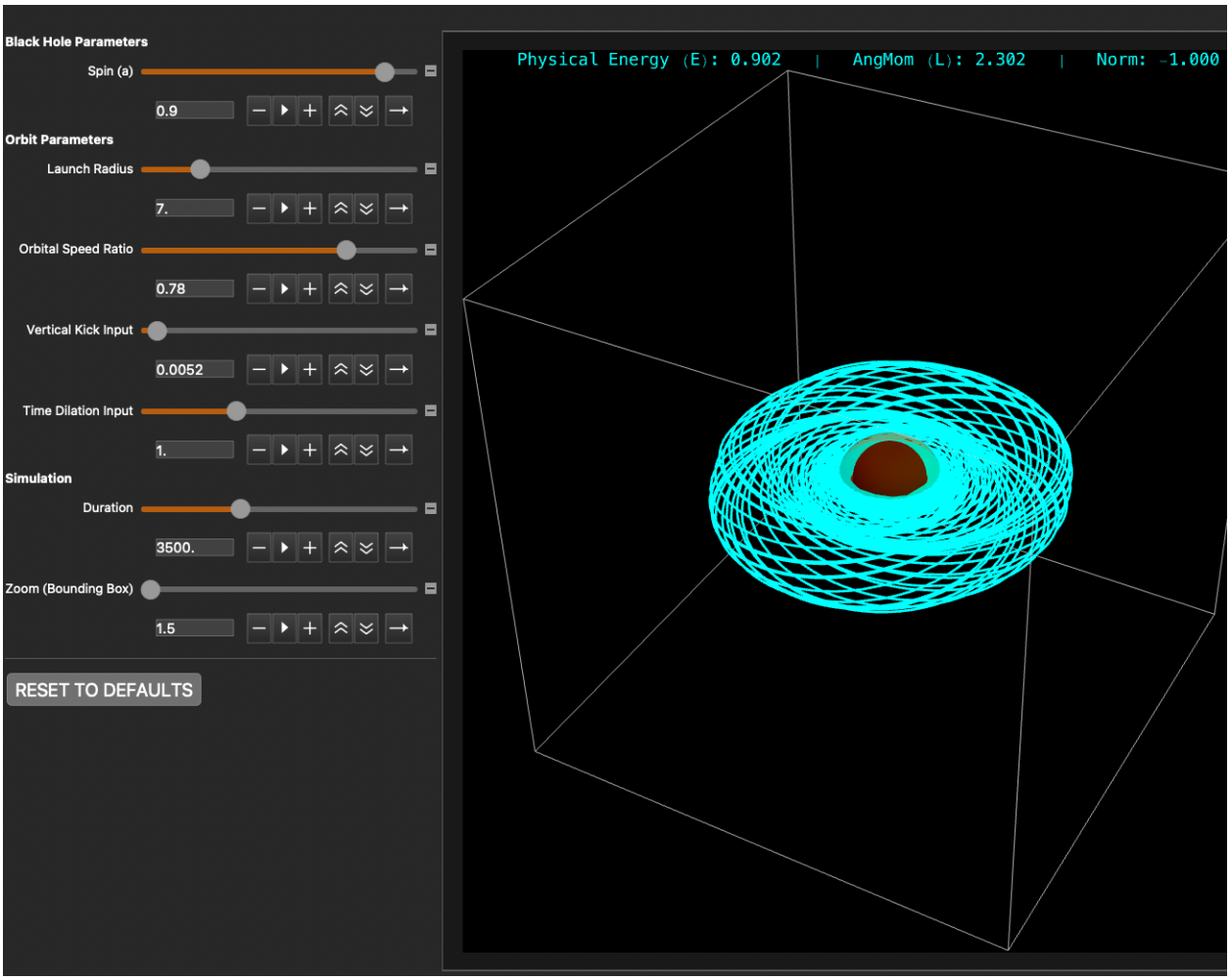
A Wizard's Quest: Spinning Black Holes in $\mathcal{UD}$

Brian Beckman

Feb 2026




Black Hole Parameters
Spin (a)
0.99
Orbit Parameters
Launch Radius
3
Orbital Speed Ratio
0
Vertical Kick Input
0
Time Dilation Input
1.
Simulation
Duration
30.
Zoom (Bounding Box)
1.5
RESET TO DEFAULTS
Physical Energy ⟨E⟩: 0.577 | AngMom ⟨L⟩: -1.143 | Norm: -1.00



Abstract

The Kerr metric around a spinning black hole is much more difficult than the Schwarzschild metric. The quest to find and solve its field equations took some 50 years from Einstein's publishing of general relativity in 1915. The *dramatis personae* include Boyer, Lindquist, Kerr, Ernst, and others who published results from 1965 to 1968. We tell the story not historically, but as a fable of challenge and victory, in a spirit of rigorous fun. Along the way, we stress-test $\mathcal{UD}$ and the Relativity Toolkit, v1.10.0. That's the novel contribution of this notebook.

Unlike static Schwarzschild, Kerr has off-diagonal terms that couple time and rotation: ***frame-dragging***. Light cones near the black hole are tipped in the spin direction, dragging orbits out of the Keplerian plane. The effect is also called *gravitomagnetism*, due to its resemblance to electrodynamical magnetism, both producing stresses perpendicular to planes of motion.

Over time, stable Kerr orbits trace out toroidal structures via nodal precession similar to that of a Lagrange top. This precession is additional to precession of the perihelion, present even in Schwarzschild (non-rotating) geometry. Kerr black holes also have *Ergospheres*, aka *static limits*,

spacetime boundaries outside the event horizons. Inside an Ergosphere, spacetime itself flows faster than light. Inflowing geodesics are swept along the spinning direction of the black hole.

Introduction: A Quest

The Wizard Chandra reveals a quest: "I have an ancient artifact, a wondrous machine, more wondrous than Antikythera. I have partially translated its name: "Visualizer." From what else I can gather, it shows orbits around a spinning black hole. I can make it do some things, but I can't operate it proficiently. Find out anything you can about it. Needless to say, I also don't know how it works. Is it beyond hope to find the blueprints? Without them, I can't replicate, maintain, or repair the machine. If it breaks, it's lost forever! Let me show you what little I know."

He shows us: The 3D Spirograph*, wherein orbits fly up and down, out of the plane, but never escape a donut-shaped envelope;* The Hula Hoop*, wherein orbits become Schwarzschild—planar—when the black hole stops spinning;* The Rip Tide*, wherein stars dumped straight down into the black hole touch the Ergosphere and are swept aside and flung away.*

We look inside the machine. The clockwork is an indecipherable bloc of definitions. Our quest is to explain them.

To do that, the wizard says, we must find, decipher, and translate the keys that lie in forgotten tomes in a Great Library fabled to be at the end of a long road. The wizard gives us a partial map, a linearized far-field approximation.

The wizard warns us that a dragon named Ansatz Kerr *blocks the road. To look the ansatz field equations in the eye will drive a knight to madness. But, the wizard gives us pieces of a Magical Sword,* Ernst*, that he thinks might help us. These pieces are new compiler functions in the 𝒰𝒟 Relativity Toolkit. We write down more of the wizard's guide for our quest journey:*

- **The Dragon**: We write the Einstein vacuum field equations, Ricci = 0, over an ansatz metric of four diagonal components, as we did with Schwarzschild, but this time including a fifth, coupling term, conjured intuitively, in the two, symmetrical, t—ϕ slots.
- **The Thorny Thicket**: While the equations are easy for 𝒰𝒟 to write, they are a stupendously large, thorny thicket of algebra on either side of the dragon, with 300-800 terms each. *No opening has ever been found, the wizard tells us.* There is no obvious structure amongst the enormity, no starting point for teasing the equations apart and solving them incrementally. (TODO: perhaps AI can take a stab at solving them; I didn't try it.)
- **The Wizard's Map**: *Are we on the right road?* The map is a new, simplified, linearized, weak-field, far-field ansatz that yields Newton-Kepler with light frame-dragging. *The map will show us when we are on the right road, with the promise that the keys we seek lie beyond the dragon. But we can't slay the dragon, and we can't step around him because he is surrounded by the impenetrable thicket as far as the eye can see.*

- **The Magical Sword**: *a sword named* Ernst *may help us. If we can find the jewels in the thicket, we can slot them into the empty hilt of the sword and, by shining sunlight through them, enchant the monster into letting us pass.* Ernst gives us a simplified equation that the Kerr metric must satisfy. *The jewels will write the equation in 𝒰𝒟 and recapitulate Ernst's proof.*
- **The Road Beyond**: Now with the full metric revealed, we compare it to the Wolfram Function Repository (WFR) and find a match. *This will be the detailed map to the Great Library.*
- **The Quest Achieved**: Should we succeed, we will find the vacuum equations now tractable, symbolically and numerically, and show how they're built into the Visualizer. We now can explain, in detail, The 3D Spirograph, The Hula Hoop, The Rip Tide.

Before we set out, the wizard fully provisions us for the quest.

Basic Provisions: the Relativity Toolkit

Supplies for the journey ahead as we search for the keys to The Wondrous Machine.

Optional: Quit for a Fresh Kernel

- *Note, if you call* `Quit` *in a notebook, evaluation may hang. If you want to "Evaluate Notebook," mark the following two cells "Evaluatable" in the Cell→Property menu, call* `Quit` *and* `FrontEndTokenExecute` *once each, then comment out the cells, or mark them unevaluatable again. Alternatively, you can quit the kernel via the Evaluation menu (last item). At that point, you can "Evaluate Notebook" in a clean environment, or evaluate cells individually.*

```
Quit[];
```

In[•]:=
```
FrontEndTokenExecute["DeleteGeneratedCells"]
```

Download the Code

In[1]:=
```
RTbranch = "main";
RTbase = "https://raw.githubusercontent.com/rebcabin/RelativityToolkit/" <>
    RTbranch <> "/";
RTbust = "?t=" <> ToString[Round[AbsoluteTime[]]];(*Force fresh download*)
RTrules = RTbase <> "RelativityToolkit.wl" <> RTbust;
RTtests = RTbase <> "RelativityToolkit_RegressionTests.wl" <> RTbust;
RTviz = RTbase <> "RelativityToolkit_VisualShowcase.wl" <> RTbust;
Get[RTrules];
```

» Relativity Toolkit version : 1.10.0

» Relativity Connection Symbol : Γ

Provisions for the Magical Sword: New Compiler Functions

The Wizard Chandra gave us the pieces for Ernst, *The Magical Sword. We are sure we will need it, but we*

should not assemble it too early, lest we damage it. Furthermore, there is little point, as its magic requires the lost jewels we must find before anything else to enchant the dragon.

We add several new general-purpose functions to the $\mathcal{UD}$ compiler corpus. This section sketches their functionality, and the rest of the notebook illustrates their use by example. These functions pertain to future problems in general relativity and differential geometry.

In this notebook, "matrix" is a synonym for a Wolfram 2D array. A tensor is either a matrix that satisfies coordinate-transformation rules, or a collection of symbolic $\mathcal{UD}$ expressions that also satisfy coordinate-transformation rules.

A Summoning Spell: Γ-getter

Just as **CalculateRicciComponent**, already in the toolkit, needs a Riemann-getter for fetching components from Riemann, so **CalculateRiemannComponent** needs a Γ-getter for fetching components of the Christoffel symbols. Heretofore, Γ-getters were ad-hoc, constructed on-the-fly. But now we offer a way to build them from rule-patterned ancestors: the metric and its inverse.

Given symbols and rules for the covariant metric, $g_{\mu\nu}$, and its contravariant inverse, $g^{\mu\nu}$, **CalculateGammaComponent** is a general way to fetch an individual component at indices $\mathcal{U}[s]$, $\mathcal{D}[m]$, $\mathcal{D}[n]$ of the Christoffel symbols. This definition and implementation mirrors its partners, **CalculateRiemannComponent** and **CalculateRicciComponent**, introduced in v1.9.0.

- *The covariant and contravariant metrics stand in a mutually matrix-inverse relationship because $g^{\mu\rho}\, g_{\rho n} = \delta^{\mu}{}_{\nu}$, the Kronecker delta, aka the Identity matrix.*

In[•]:=
```
CalculateGammaComponent[s_, m_, n_,
   gDD_Symbol, gDDRules_,
   gInvUU_Symbol, gInvUURules_,
   coords_] :=
  (ContractAll[
       ChristoffelsFromMetric[gDD, gInvUU, s, m, n],
       coords] /.
     gDDRules /.
    gInvUURules) //
   EvaluateUDPartials;
```

The Handguard: Gradient

For Ernst's Equation (the Magical Sword), we'll need the gradient operator in curved spacetime.

$$\nabla^{\mu} f \equiv g^{\mu\nu} f_{,\nu} \tag{1}$$

- *It is also the workhorse of Hamilton-Jacobi Theory, whence we get the geodesic equations, e.g.*

$$H = \frac{1}{2}\, g^{\mu\nu} p_{\mu} p_{\nu} \quad , \text{ where } p_{\mu} = S_{,\mu} \text{ is the derivative of the action } S \tag{2}$$

Because **GradRaised** needs only the contravariant metric, it takes only a symbol and rules for that.

```
In[•]:= GradRaised[func_, gInvUU_Symbol, gInvUURules_, coords_] :=
          Table[Sum[(gInvUU[𝒰[μ], 𝒰[ν]] /. gInvUURules) D[func, ν],
            {ν, coords}], {μ, coords}];
```

The Hilt: Gradient Squared

The hilt of the Magical Sword, with the empty slots for the jewels, requires the squared gradient:

$$(\nabla f)^2 \equiv \nabla_\mu f \, \nabla^\mu f \equiv g^{\mu\nu} f_{,\mu} f_{,\nu} \equiv \nabla f \bullet \nabla f \tag{3}$$

Like **GradientRaised**, GradSquared needs only the contravariant metric, taking only a symbol and rules for that.

```
GradSquared[func_, gInvUU_Symbol, gInvUURules_, coords_] :=
  With[{grad = MakeIndexer[
      GradRaised[func, gInvUU, gInvUURules, coords], coords]},
   Simplify[
    Sum[D[func, μ] × grad[μ], {μ, coords}]]];
```

The Blade: Scalar Laplacian

The blade of the Magical Sword is the Laplacian:

$$\nabla^2 f \equiv \frac{1}{\sqrt{|g|}} \partial_\mu \left(\sqrt{|g|} \, g^{\mu\nu} \partial_\nu f \right) \tag{4}$$

- *This is not* $(\nabla f)^2$.

In addition to the now-regulatory contravariant metric, the Laplacian needs one more parameter, the square root of the determinant of the **covariant** metric, $g_{\mu\nu}$—Emphasis: the determinant of the down-down metric. This is best computed externally and passed in.

```
In[•]:= ScalarLaplacian[func_, gInvUU_Symbol, gInvUURules_, sqrtDetg_, coords_] :=
          With[{grad = MakeIndexer[GradRaised[func, gInvUU, gInvUURules, coords], coords]},
           Simplify[
            Sum[D[sqrtDetg * grad[μ], μ], {μ, coords}] / sqrtDetg]];
```

The Wondrous Machine: Kerr Orbit Visualizer

The wizard has translated part of a warning label on the Wondrous Machine: "this ancient artifact is designed to illustrate physically plausible scenarios. Given the particular decomposition of initial conditions and parameters, it is possible to create non-physical scenarios such as stars that travel faster than the speed of light. When this happens, a red light may appear in the Heads-Up Display.

Dependencies, Mute, En-Bloc

The quest should reveal the meaning of these definitions, but the wizard cannot translate them. For now, take them as granted, as just enough to enable the Visualizer.

In[8]:=
```
coords = {t, r, θ, ϕ};
Σ = r^2 + a^2 Cos[θ]^2;
Δ = r^2 - 2 M r + a^2;
kerrMetric = {{-(1 - 2 M r/Σ), 0, 0, -2 M a r Sin[θ]^2/Σ},
   {0, Σ/Δ, 0, 0},
   {0, 0, Σ, 0},
   {-2 M a r Sin[θ]^2/Σ, 0, 0, (r^2 + a^2 + 2 M a^2 r Sin[θ]^2/Σ) Sin[θ]^2}};
Clear[gDD, gUU];
kerrRules = Join[
 MatrixToUDRules[kerrMetric, gDD, 𝒟, coords],
 {gDD[__] → 0}];
kerrInverse = Simplify[Inverse[kerrMetric]];
kerrInvRules = Join[
 MatrixToUDRules[kerrInverse, gUU, 𝒰, coords],
 {gUU[__] → 0}];
kerrGamma[s_, m_, n_] :=
  CalculateGammaComponent[s, m, n, gDD, kerrRules, gUU, kerrInvRules, coords];
```

Unknown functions of proper time τ

In[17]:=
```
funcs = {t[τ], r[τ], θ[τ], ϕ[τ]};
velocities = D[funcs, τ];
accelerations = D[velocities, τ];
```

The Geodesic Equation, $x'' + \Gamma\, x'\, x' = 0$

In[20]:=
```
geodesicEqs =
  With[{velLookup = MakeIndexer[velocities, coords], (* toolkit function *)
    accLookup = MakeIndexer[accelerations, coords]},
   Table[accLookup[μ] +
      Sum[(kerrGamma[μ, ν, λ] /. Thread[coords → funcs]) velLookup[ν] × velLookup[λ],
       {ν, coords},
       {λ, coords}] == 0,
     {μ, coords}]];
```

Explicit Acceleration Rules

In[21]:=
```
explicitAccelerations = Solve[
     geodesicEqs, accelerations][[1]] /.
   Rule → Equal;
```

The Visualizer itself

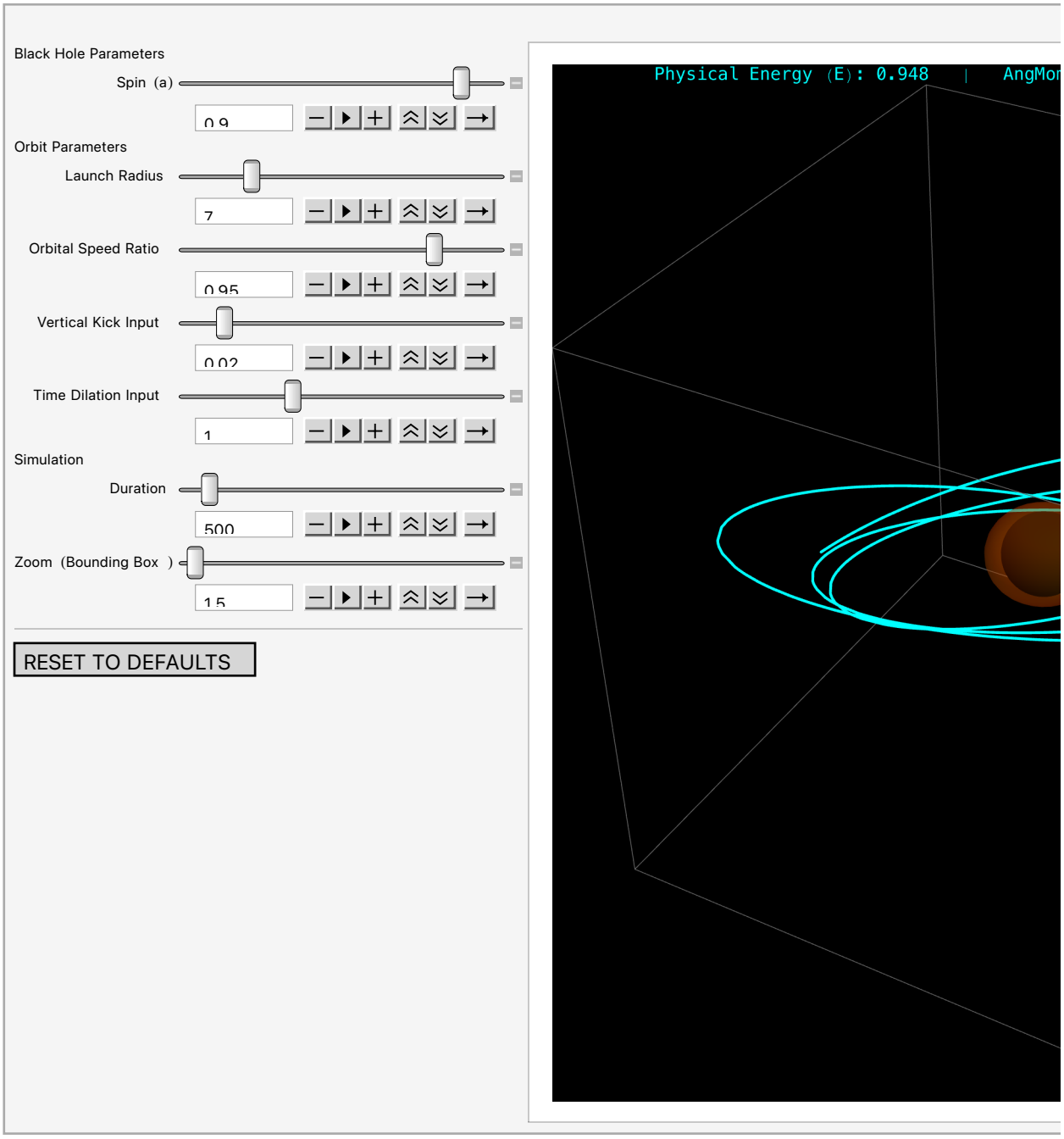
In[22]:=
```
With[{
  defaultSpinParameter = 0.9,
```

```
 defaultInitialRadius = 7.0,
 defaultAngMomRatio = 0.95, (*1.0 means "100% of Circular Orbit Speed"*)
 defaultVerticalKick = 0.02,
 defaultEparam = 1.0,
 defaultDuration = 500,
 defaultPuff = 1.5},
{minSpin = 0.0, maxSpin = 0.99,
 minRadius = 1.5, maxRadius = 30.0,
 minAngMomRatio = -1.5, maxAngMomRatio = 1.5, (*Universal for all radii!*)
 minKick = 0.0, maxKick = 0.20,
 minE = 0.5, maxE = 2.0,
 minDuration = 10, maxDuration = 10 000,
 minPuff = 1.5, maxPuff = 30},
{innerLimitRadius = 1.1},
Manipulate[
 Module[{sol, Mval = 1, Tmax = duration, rMin, rawU, metricAtStart,
   rawNorm, scaleFactor, finalU, physicalE, physicalL, isPhysical,
   baseOmega, calculatedInitL},
  (*BASELINE SPEED FOR CURRENT RADIUS*)
  (*Keplerian approximation:Omega=√(M/r^3) *)
  (*scale the slider automatically as Radius changes*)
  baseOmega = √(Mval / initR^3);
  (*APPLY USER RATIO*)
  (*The slider angMomRatio multiplies this baseline*)
  calculatedInitL = angMomRatio * baseOmega;
  (*SETUP METRIC & NORMALIZE*)
  metricAtStart = kerrMetric /. {r → initR, a → spinParam, θ → π / 2, M → Mval};
  (*Use the calculated L instead of a raw input*)
  rawU = {initEparam, 0, initVelTheta, calculatedInitL};
  rawNorm = rawU.metricAtStart.rawU;
  If[rawNorm < 0,
   (isPhysical = True;
    scaleFactor = 1 / Sqrt[-rawNorm];
    finalU = rawU * scaleFactor;
    physicalE =
     -(metricAtStart[[1, 1]] * finalU[[1]] + metricAtStart[[1, 4]] * finalU[[4]]);
    physicalL = metricAtStart[[4, 1]] * finalU[[1]] + metricAtStart[[4, 4]] * finalU[[4]];),
   (isPhysical = False;
    finalU = rawU;
    physicalE = 0; physicalL = 0;)];
  (*INTEGRATE*)
  initConds = {
```

```
    t[0] == 0, t'[0] == finalU[[1]],
    r[0] == initR, r'[0] == finalU[[2]],
    θ[0] == π / 2, θ'[0] == finalU[[3]],
    ϕ[0] == 0, ϕ'[0] == finalU[[4]]};
  sol = Quiet@NDSolve[{
       explicitAccelerations /. {M → Mval, a → spinParam},
       initConds} /. {M → Mval, a → spinParam},
     funcs, {τ, 0, Tmax}, Method → "StiffnessSwitching", MaxSteps → 100 000];
  rMin = Quiet@MinValue[{(r[τ] /. sol)[[1]], 0 ≤ τ ≤ Tmax}, τ];
  (*VISUALIZE*)
  Show[
   Quiet@ParametricPlot3D[{
       r[τ] * Sin[θ[τ]] * Cos[ϕ[τ]],
       r[τ] * Sin[θ[τ]] * Sin[ϕ[τ]],
       r[τ] * Cos[θ[τ]]} /. sol,
     {τ, 0, Tmax},
     PlotStyle → {If[rMin < innerLimitRadius, Red, Cyan], Thickness[0.003]},
     PlotRange → {{-initR * puff, initR * puff},
       {-initR * puff, initR * puff}, {-initR * puff, initR * puff}},
     Boxed → True, Axes → False, Background → Black, ImageSize → Large,
     PlotLabel → If[isPhysical,
       Style[Row[{
          "Physical Energy (E): ", NumberForm[physicalE, {4, 3}],
          "   |   AngMom (L): ", NumberForm[physicalL, {4, 3}],
          "   |   Norm: ", NumberForm[finalU.metricAtStart.finalU, {4, 3}]}],
        Cyan, FontFamily → "Consolas"],
       Style["UNPHYSICAL: Speed > c (Tachyon)",
        Red, Bold, FontFamily → "Consolas"]]],
   Graphics3D[{
     Black, Sphere[{0, 0, 0}, Mval + √(Mval^2 - spinParam^2)],
     ParametricPlot3D[
       With[{rE = Mval + √(Mval^2 - spinParam^2 * Cos[θ]^2)},
        rE {Sin[θ] Cos[ϕ], Sin[θ] Sin[ϕ], Cos[θ]}], {θ, 0, π}, {ϕ, 0, 2 π},
       Mesh → None, PlotStyle → {Opacity[0.3], Orange}][[1]]}]],
  (*CONTROLS*)
  Style["Black Hole Parameters", Bold],
  {{spinParam, defaultSpinParameter, "Spin (a)"},
   minSpin, maxSpin, Appearance → "Open"},
  Style["Orbit Parameters", Bold],
  {{initR, defaultInitialRadius, "Launch Radius"},
   minRadius, maxRadius, Appearance → "Open"},
```

```
{{angMomRatio, defaultAngMomRatio, "Orbital Speed Ratio"},
 minAngMomRatio, maxAngMomRatio, Appearance → "Open"},
{{initVelTheta, defaultVerticalKick, "Vertical Kick Input"},
 minKick, maxKick, Appearance → "Open"},
{{initEparam, defaultEparam, "Time Dilation Input"},
 minE, maxE, Appearance → "Open"},
Style["Simulation", Bold],
{{duration, defaultDuration, "Duration"},
 minDuration, maxDuration, Appearance → "Open"},
{{puff, defaultPuff, "Zoom (Bounding Box)"},
 minPuff, maxPuff, Appearance → "Open"}, Delimiter,
Button["RESET TO DEFAULTS",
 spinParam = defaultSpinParameter;
 initR = defaultInitialRadius;
 angMomRatio = defaultAngMomRatio;
 initVelTheta = defaultVerticalKick;
 initEparam = defaultEparam;
 duration = defaultDuration;
 puff = defaultPuff;,
 ImageSize → Medium]]]
```

Out[22]=



Kerr Visualizer User's Manual

Along with the machine, the wizard has a priceless scroll, but he cannot translate it without the keys. We, however, as omniscient narrators of the tale, can read what the wizard and the adventurers cannot. Knowing it now, we will judge the success or failure of the quest.

Units

Sliders on the Visualizer read in dimensionless quantities. To relate these quantities to the physical world, the Visualizer adopts "natural units" ($G = c = 1$). In these units, one unit of mass $\left(1 = M = c^2/G\right)$ has dimensions of Length, with conversion factors

In[23]:=
```
UnitConvert[Quantity["GravitationalConstant"] / Quantity["SpeedOfLight"]^2, "SIBase"]
```

Out[23]=

7.426×10^{-28} m/kg

In[24]:=
```
UnitConvert[Quantity["SpeedOfLight"]^2 / Quantity["GravitationalConstant"], "SIBase"]
```

Out[24]=

1.3466×10^{27} kg/m

- Mass is measured multiples of the black-hole mass M_{BH}
- Distances are measured in multiples of M_{BH}.
- Times are measured in "light-crossing time" across one BH radius, M_{BH}/c

Fundamental Constants

For brevity, define

In[25]:=
```
constG = Quantity["GravitationalConstant"];
constC = Quantity["SpeedOfLight"];
```

Mass to Geometricized Length (GM/c^2)

In[27]:=
```
MassToLength[mass_Quantity] := UnitConvert[constG * mass / constC^2, "Meters"];
```

Examples

In[28]:=
```
Echo[MassToLength[AstronomicalData["Sun", "Mass"]], "Solar Mass in meters :"];
```

» Solar Mass in meters : 1476.6 m

In[29]:=
```
Echo[UnitConvert[MassToLength[AstronomicalData["Earth", "Mass"]], "cm"],
  "Earth Mass in cm :"];
```

» Earth Mass in cm : 0.4435 cm

In[30]:=
```
Echo[gravRadSgrA = UnitConvert[4.15 × 10^6 MassToLength[Quantity["SolarMass"]], "km"],
  "Grav Radius SgrA* :"];
```

» Grav Radius SgrA* : 6.12796×10^6 km

In[31]:=
```
Echo[UnitConvert[gravRadSgrA, "AU"], "Grav Rad SgrA* :"];
```

» Grav Rad SgrA* : 0.0409629 au

In[32]:=
```
Echo[UnitConvert[
    Quantity["SemimajorAxis" /. AstronomicalData["Mercury", "OrbitRules"], "m"],
    "AU"], "Orbit Rad of Mercury in AU :"];
```

» Orbit Rad of Mercury in AU : 0.38709893 au

Spin Slider ($a = \frac{J}{Mc}$)

Angular momentum is measured in meters. The slider measures *specific spin angular momentum* of the black hole, normalized to the mass (in meters) of the black hole. Larger values mean faster spinning.

- physical definitions: $a = \frac{J}{Mc}$, units of length = $\left(\frac{\text{Joule}\cdot s}{\text{kg}\cdot m/s} = \frac{\text{kg}\cdot m^2\cdot s/s^2}{\text{kg}\cdot m/s} = m\right)$.
- visualizer input: dimensionless ratio a/M
- range: 0 (Schwarzschild) to 1 (Extreme Kerr).
  - a is bounded above by 1 because the event horizon is at radius $M\sqrt{1-a^2}$. $a > 1$ yields a pure imaginary radius event horizon, which supposedly drops out of existence and exposes a *naked singularity*. At $a = 1$, frame-dragging reaches the speed of light, and the event horizon is supposedly stripped away by centrifugal force.

Example: The Sun

The Sun is a slow rotator. What if it were a black hole? What value should we put on the *Spin* slider?

In[33]:=
```
solM = Echo[AstronomicalData["Sun", "Mass"], "M⊙ ="];
solR = Echo[AstronomicalData["Sun", "Radius"], "R⊙ ="];
solPeriod = Echo[AstronomicalData["Sun", "RotationPeriod"], "P⊙ ="];
```

» $M_\odot$ = 1.988435 × 10^{30} kg

» $R_\odot$ = 6.955 × 10^{8} m

» $P_\odot$ = 2.164 × 10^{6} s

The coefficient in the moment-of-inertia formula for a centrally condensed star like the Sun is about 0.070, much less than the 2/5 for a uniform sphere.

In[36]:=
```
solMICoefficient = 0.070;
solMomentOfInertia = Echo[solMICoefficient * solM * solR^2, "MI⊙ :"];
solω = Echo[2 π / solPeriod, "ω⊙ ="];
solJ = Echo[UnitConvert[solMomentOfInertia * solω, "SI"], "J⊙ ="];
solJ = Echo[UnitConvert[solMomentOfInertia * solω, "SIBase"], "J⊙ ="];
```

» $MI_\odot$: 6.73292 × 10^{46} kg m^2

» $\omega_\odot$ = 2.904 × 10^{-6} per second

» $J_\odot$ = 8.73525 × 10^{18} kg au^2/s

» $J_\odot$ = 1.95491 × 10^{41} kg m^2/s

In[41]:=
```
AngMomToSpin[J_Quantity, M_Quantity] := UnitConvert[J / (M * constC), "Meters"];
```

In[42]:=
```
solGeomM = Echo[MassToLength[solM], "M⊙[L] ="];
solGeomA = Echo[AngMomToSpin[solJ, solM], "J⊙[L] ="];
```

» $M_\odot$[L] = 1476.6 m

» $J_\odot$[L] = 327.94 m

Spin slider input (Dimensionless a/M)

In[44]:=
```
solSpinParam = solGeomA / solGeomM
```

Out[44]=
```
0.222085
```

Launch Radius Slider (r/M)

The starting distance of the test particle from the center of the black hole, aka the singularity at the center of the coordinate system.

- physical definition: Coordinate distance in Boyer-Lindquist coordinates.
- visualizer input: Multiples of the Gravitational Radius (GM/c^2).

Example: Sgr A*, supermassive black hole at center of the Milky Way

In[45]:=
```
Echo[sgrMass = Quantity[4.15*^6, "SolarMass"], "MSgrA* ="];
Echo[sgrSpinParam = 0.90, "(a/M)SgrA* ="];
```

» M_{SgrA*} = 4.15×10^6 $M_\odot$

» $(a/M)_{SgrA*}$ = 0.9

In[47]:=
```
Echo[sgrGeomM = MassToLength[sgrMass], "MSgrA*[L] ="];
Echo[UnitConvert[2 * sgrGeomM, "AstronomicalUnit"],
  "(event horizon EH) 2 MSgrA* ="];
Echo[UnitConvert[solR, "AstronomicalUnit"], "M⊙[AU] ="];
Echo[2 * sgrGeomM / solR, "SgrA* EH in solar radii: "];
```

» M_{SgrA*}[L] = 6.12796×10^9 m

» (event horizon EH) 2 M_{SgrA*} = 0.0819258 au

» $M_\odot$[AU] = 0.004649 au

» SgrA* EH in solar radii: 17.6217

If we orbited Sgr A* at 1 AU, what should *Launch Radius* be?

In[51]:=
```
Echo[orbitDist = Quantity[1, "AstronomicalUnit"], "Orbit distance ="];
sgrInitR = orbitDist / sgrGeomM
```

» Orbit distance = au

Out[52]=
```
24.4123
```

An Independent Computation

Orbit SgrA* at a coordinate distance of 1 AU

Gravitational Radius of SgrA*

In[53]:=
```
Echo[gravRadSgrA = UnitConvert[4.15 × 10^6 MassToLength[Quantity["SolarMass"]], "km"],
  "Grav Radius SgrA* :"];
```

» Grav Radius SgrA* : 6.12796 × 10^6 km

In[54]:=
```
Echo[au = UnitConvert[Quantity["AstronomicalUnit"] // N, "km"],
  "astronomical unit :"];
```

» astronomical unit : 1.49598 × 10^8 km

Launch Radius Slider Value

In[55]:=
```
au / gravRadSgrA
```

Out[55]=

24.4123

Time Dilation and the Heads-Up Display

The “Time Dilation Input” slider controls the ratio of time flow to spatial motion. It’s a “mix knob” between “moving through time” and “moving through space.” For most stable orbits, leave this slider at 1.0. The physics engine will automatically calculate the true time dilation (Physical Energy E) required to keep the particle normalized ($u.u = -1$). Both E and the calculated norm appear in the Heads-Up Display (HUD) that labels the plots. Unphysical norms are possible for some slider settings overall, in which case, the HUD turns red.

This controls the “Time Component” of the launch vector. It acts as a mix knob between “Moving through Time” and “Moving through Space.”

1.0 is the standard setting.

To slow down (make heavier), increase the slider (> 1.0). Why? Putting more of the velocity vector into “time,” the particle moves through time faster, but now through space slower because the norm $(-\text{time}^2 + \text{space}^2)$ is constant. The particle acts more heavy and sluggish.

To speed up (make lighter): decrease the slider (< 1.0). Why? Reducing the “time” component. To maintain normalization, the “space” components must increase. The particle gains kinetic energy.

Tip: If your particle flies off into space, i.e., becomes unbound, it has too much energy. Increase the time dilation to calm it down.

The calculators below let you predict what you should see in the HUD for various scenarios. Use them to verify your intuition and the physics of the simulation!

CALCULATOR 1: Relativistic Flyby, aka the Lorentz Factor

In a high-speed flyby far from the black hole, gravity is weak. The HUD Energy E should be close to the γ of special relativity, $1 \Big/ \sqrt{1 - v^2}$.

In[56]:=
```
FlybySlider[velocity_Quantity] :=
  Module[{beta, gamma},
   beta = QuantityMagnitude[velocity / constC];
   gamma = 1 / Sqrt[1 - beta^2];
   gamma];
```

CALCULATOR 2: Stable Circular Orbit, aka Gravitational Time Dilation

Deep in the gravity well, time slows down significantly. Even if the Time Dilation input slider is 1.0, the HUD will show a much higher Energy (u^t) to balance the intense gravitation.

In[57]:=
```
CircularOrbitSlider[radiusM_] := 1 / Sqrt[1 - 3 / radiusM];
```

Example: Interstellar Visitor

A probe flies by at 20% the speed of light.

In[58]:=
```
Echo[probeSpeed = Quantity[0.2, "SpeedOfLight"], "probe speed :"];
Echo[predictedHUD = FlybySlider[probeSpeed], "predicted HUD E :"];
```

» probe speed : 0.2 c

» predicted HUD *E* : 1.02062

Example: Parking Orbit

Park a particle in a circular orbit at radius $r = 10\,M_{BH}$.

In[60]:=
```
Echo[targetRadius = 10.0, "target radius :"];
Echo[predictedHUD = CircularOrbitSlider[targetRadius], "predicted HUD E :"];
```

» target radius : 10.

» predicted HUD *E* : 1.19523

The auto-normalizer automatically boosts u^t to this value.

Example: Innermost Stable Orbit (ISCO)

The closest possible orbit, according to Schwarzschild, is at $r = 6\,M_{BH}$.

In[62]:=
```
Echo[iscoRadius = 6.0, "ISCO radius :"];
Echo[predictedHUD = CircularOrbitSlider[iscoRadius], "predicted HUD E :"];
```

» ISCO radius : 6.

» predicted HUD *E* : 1.41421

Time is flowing 41% faster for us at a safe distance than for the astronaut in orbit.

Orbital Speed Ratio Slider

The “Orbital Speed Ratio” slider is calibrated to the Keplerian circular orbital speed for the chosen Launch Radius. The Visualizer does not accept independent settings for orbital radius and for orbital angular momentum, due to unusable sensitivity of such controls. Instead, the user specifies desired

orbital angular momentum for a scenario as a ratio of desired speed to a Keplerian norm.

- *Orbital angular momentum is the angular momentum of the test particle or star or spaceship orbiting the black hole, and is distinct from and independent of the spin parameter, which sets the angular momentum of the black hole itself.*
- Setting = 1.0: Approximately circular orbit (Keplerian speed).
- Setting < 1.0: The particle falls inward due to insufficient centrifugal force. It may be desirable to visualize
- Setting > 1.0: The particle climbs outward due to excess centrifugal force.

Below,we calculate 1.0 in km/*s* for various various orbits.

CALCULATOR: Keplerian Orbital Velocity $\left(\sqrt{GM/r}\right)$

In[64]:=
```
KeplerVelocity[mass_Quantity, radius_Quantity] :=
  UnitConvert[ √(constG * mass / radius) , "Kilometers" / "Seconds"];
```

Example: Earth's Orbit Around the Sun

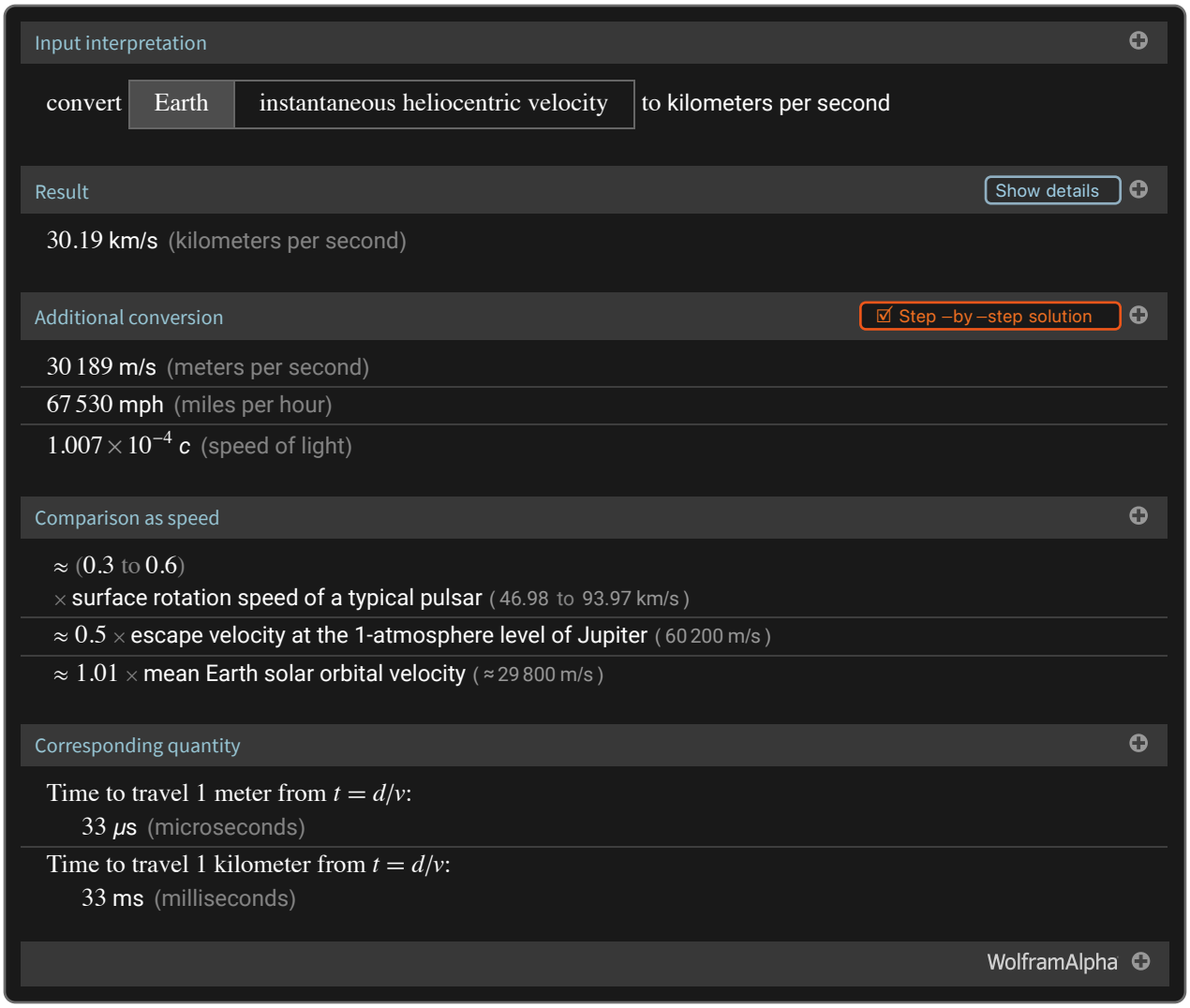
In[65]:=
```
Echo[vEarth = KeplerVelocity[solM, Quantity[1, "AstronomicalUnit"]],
  "Earth's Obital Speed :"];
```

» Earth's Obital Speed : 29.785 km/s

Not far from Wolfram-Alpha's answer:

In[66]:=

Earth's orbital speed around the Sun in km/s



In the Visualizer, setting Ratio to 1.0 achieves this balance automatically.

Example: Sgr A*, Supermassive Black Hole in the Milky Way

What's the speed if we orbit Sgr A* at 1 AU's distance? This is not for a slider setting, just for information.

In[67]:=
```
Echo[vSgrA = KeplerVelocity[Quantity[4.15*^6, "SolarMass"],
    Quantity[1, "AstronomicalUnit"]], "Orbital Speed for SgrA* :"];
```

» Orbital Speed for SgrA* : 60 675.9 km/s

2000 times the speed Earth needs around the Sun to keep from falling in.

Vertical Kick Input Slider, aka Inclination

The *Vertical Kick* slider imparts an initial velocity in the longitudinal θ direction, perpendicular to the equatorial plane. Physically, this creates *orbital inclination* or tilt.

4. Vertical Kick Input `initVelTheta` (u^θ)

This is the initial velocity perpendicular to the equatorial plane.

- physical definition: $d\theta/d\tau$, in radians per unit proper time.
- Setting = 0: Equatorial Orbit.The particle stays in a flat plane.
- Setting > 0.0: Inclined Orbit.The particle bobs up and down if spin > 0

With Schwarzschild (non-spinning) black holes, an inclined orbit is just a flat tilted ring. But with Kerr (spinning) black holes, frame dragging causes the entire orbital plane to precess.This creates the spirographical donut shapes you see in the Visualizer. The larger this kick,the "thicker" the donut.

This is the initial velocity perpendicular to the equatorial plane.

CALCULATOR: Kick to Inclination Angle (Approx)

This is a rough heuristic for the Visualizer's behavior.

In[68]:=
```
InclinationEstimator[kickInput_, angMomRatio_] :=
  Module[{angle = ArcTan[kickInput / angMomRatio] * 180 / π},
   Quantity[angle, "AngularDegrees"]];
```

Example: Spirograph

In[69]:=
```
Echo[myKick = 0.20, "θ kick :"];
Echo[myRatio = 1.0, "Ang Mom Ratio :"];
Echo[estAngle = InclinationEstimator[myKick, myRatio],
  "Estimated Inclination angle (deg) :"];
```

» θ kick : 0.2

» Ang Mom Ratio : 1.

» Estimated Inclination angle (deg) : 11.3099°

This tilt lets the black hole's spin twist the orbit into a torus.

Duration Slider

How long is'Duration=100' in human time?

The simulation time unit is the light-crossing time of the mass radius (M/c). Since our simulation tracks the astronaut's journey, this 'Duration' represents Proper Time—the time elapsed on the astronaut's watch.

In[72]:=
```
timeUnitSun = Echo[UnitConvert[solGeomM / constC, "Seconds"], "T⊙ (time tick) ="];
timeUnitSgr = Echo[UnitConvert[sgrGeomM / constC, "Seconds"], "TSgrA* (time tick) ="];
```

» $T_\odot$ (time tick) = 4.926×10^{-6} s

» T_{SgrA*} (time tick) = 20.4407 s

Example: Duration 1000

How long, in proper time, does a duration setting of 1000 last? Because of time dilation, a journey of

duration = 1000 for the astronaut might take much longer for a distant observer. The Visualizer plots the path for a set amount of time experienced by an astronaut riding on the path.

In[140]:=
```
Echo[UnitConvert[1000 * timeUnitSun, "Seconds"],
  "1000 sim ticks around the Sun in proper time is :"];
Echo[UnitConvert[1000 * timeUnitSgr, "Hours"],
  "1000 sim ticks around Sag A* in proper time is :"];
```
» 1000 sim ticks around the Sun in proper time is : 0.004926 s

» 1000 sim ticks around Sag A* in proper time is : 5.67797 h

The Wizard's Experiments

Here are the settings that the Wizard knows. Neither he nor the adventurers know what they mean in any depth, but we, the omniscient narrators, are privileged to know now.

The 3D Spirograph (Toroidal Precession)

In the (+) mark upper right of the Manipulate box, choose the bookmark "The 3D Spirograph." Alternatively, set parameters as follows

- Spin: 0.9 (**default**; high spin emphasizes the effect)
- Initial Radius: 7.0 (**default**)
- Orbital Speed Ratio: 0.78 (**less than default**)
- Vertical Kick: 0.0052 (**less than default**)
- Time Dilation: 1.0 (**default**)
- Duration: 3500 **(try animating slowly** (▷) from duration 0 and 12-16 clicks on the down speed control**)**
- Bounding Box Scale: 1.5 (**default**)

In this bookmarked visualization, a particle traces a donut-shaped shell around the black hole. This is the Lense-Thirring Effect at moderate setting. The phenomenon changing the osculating plane is nodal precession.

The Setup: The black hole is spinning rapidly (a = 0.9). The particle has small slight vertical kick (v_θ = 0.0052) out of the equatorial plane.

The Result: With a non-rotating Schwarzschild black hole, this orbit would just be a precessing ellipse that stays on a single flat plane. But here, the black hole's spin drags the entire space around with it, adding nodal precession.

As the particle orbits, the osculating plane (instantaneous plane of travel) is continuously twisted by the black hole's spin. The nodes, where the orbit crosses the equator, circulate over time, causing the orbit to precess not just in the radial direction, like Mercury's perihelion precession around the Sun, but in azimuth, painting a complex torus (donut) shape instead of just a ring. Near a spinning black hole,

there is no such thing as a fixed orbital plane.

The Hula Hoop (Schwarzschild Limit)

In the (+) mark upper right of the Manipulate box, choose the bookmark “The Hula Hoop.” Alternatively, set parameters as follows

- Spin: 0.0 (**the critical change**)
- Initial Radius: 7.0 (**default**)
- Orbital Speed Ratio: 1.09 (**change**)
- Vertical Kick: 0.0078 (**change**)
- Time Dilation: 1 (**change**)
- Duration: 6000 (**change**)
- Bounding Box Scale: 4 (**change**)

The donut, and the apparent chaos, disappears. Try animating **spin slowly** (10 clicks on the down-speed control for spin).

The Rip Tide

In the (+) mark upper right of the Manipulate box, choose the bookmark “The Rip Tide.” Alternatively, set parameters as follows

- Spin: 0.99 (**max**)
- Initial Radius: 3 (**change**)
- Orbital Speed Ratio: 0 (**change**)
- Vertical Kick: 0 (**change**)
- Time Dilation: 1.0 (**default**)
- Duration: 30 (**change**)
- Bounding Box Scale: 1.5 (**default**)

In this bookmarked visualization, we have a situation where Newtonian intuition breaks down.

The Setup: a test particle at radius 1.8, inside the orange Ergosphere ($r < 2.0$) but outside the black event horizon ($r \approx 1.14$).

The Drop: zero angular momentum (Ang Mom = 0). In a Newtonian universe (or far from the black hole), a rock droped zero angular momentum falls straight into the center of gravity, like a stone dropped from a tower.

The Deadly Fate: The red trail does not fall straight in. Instead, it’s violently ripped around the black hole in a spiral before plunging in.

This illustrates the Lense-Thirring effect (frame-dragging) at its extreme.

Inside the orange shell—the Ergosphere—the spin of the black hole drags spacetime along with it so

fast that space itself move at the speed of light relative to a distant observer.

The boundary of the Ergosphere is the ***static limit***. Inside this boundary, it is impossible to remain "static," at a constant longitude ϕ. To stand still here would require moving against the flow of space faster than light.

Even though the particle has zero angular momentum—it's trying to fall straight in—any *coordinate grid* the particle lives on is rotating. The effect is intrinsic to the geometry. No non-singular coordinate system (gauge choice) can get rid of it. The particle is swept downstream by the rip tide of spacetime. It is forced to co-rotate with the black hole, no matter how hard it resists.

The code turns the track red because the particle is inside the danger zone ($r < 2.1$). It shows the particle being digested—forced into rotation and eventually swallowed beyond the horizon.

The Dragon: Kerr Metric *ab-initio*

The Wizard Chandra warned us: "Do not look the dragon, Ansatz Kerr, in the eye, for his gaze is madness." But being brave (foolhardy?), we go straight into the breach. The elder dragon, Ansatz Schwarzschild, yielded to a frontal attack, after all. Why not Ansatz Kerr?

We make a trap—an ansatz metric. The beast is stationary, sleeping, unchanging in time. He is also axisymmetric, appearing the same from any angle around its spin axis. For the other sleeping dragon, Ansatz Schwarzschild, a diagonal metric sufficed. But this dragon, Ansatz Kerr, spins. To trap it, we must add a twist: a cross-term, $E_{\text{rot}}(r, \theta)$, linking time t and azimuth ϕ. This is the mathematical embodiment of frame-dragging.

We cast our net (the ansatz metric) and bind it with the Einstein vacuum field equations: $R_{\mu\nu} = 0$.

The $\mathcal{UD}$ Toolkit*, faithful squire, does the dirty work. It calculates the Christoffel symbols and the Riemann curvature. But when asked for* $R_{\mu\nu}$*—the Ricci tensor, the final binding spell—the dragon awakens, roars, and breathes fire. The resulting equations are monstrosities. A single component,* R_{rr}*, swells to hundreds of terms. This is the thorny thicket of algebra with no obvious opening to simplification.*

We have found the dragon, but we cannot slay it and cannot step through the thicket around him. The equations are too dense, too coupled, and too angry to yield a solution by force. We need a better plan. We need the Wizard's Map and the Magical Sword.

Here is how we got the thicket in the first place—our first attempt at finding a metric for a spinning black hole:

- Set up ansatz for a stationary, axisymmetric metric.
- Write ***vacuum field equations***, Ricci = 0, via $\mathcal{UD}$.
  - Sub-strategy: Calculate Ricci components from toolkit functions `ChristoffelsFromMetric` and `CalculateRiemannComponent`.
  - Unlike Schwarzschild, find that the Ricci equations are unwieldy: it's not at all clear how to drive Wolfram to produce a result.

- Symbolically demonstrate Newtonian gravitation and mild gravitomagnetism in the weak-field limit, when the orbiting star is far from the black hole.
- Follow a historical derivation of the full solution: the Ernst Equation.

Stationary Axisymmetric Ansatz

In[76]:=
```
coords = {t, r, θ, ϕ};
```

Set up unknown functions of r and θ only, meaning of "stationary and axisymmetric," i.e., doesn't depend on time t or azimuth ϕ. Assume we can find coordinate functions r and θ where the r—θ sector is diagonal (***Boyer-Lindquist coordinates***; see References)

- $A(r, \theta)$: time potential
- $B(r, \theta)$: radial metric
- $C(r, \theta)$: latitudinal metric
- $D(r, \theta)$: azimuthal, metric
- $E(r, \theta)$: rotational t—ϕ coupling / frame-dragging, the signature component of Kerr over Schwarzschild.

Keep an eye on E_{rot}.

Pick convenient names because C and D are reserved by Wolfram and E is ambiguous

In[77]:=
```
params = {Ak[r, θ], Bk[r, θ], Ck[r, θ], Dk[r, θ], Erot[r, θ]};
```

Metric as a matrix; under the (–1, 1, 1, 1) sign convention: time-coupled components are negative.

In[78]:=
```
ansatzMetric = ⎛ -Ak[r, θ]      0         0      -Erot[r, θ] ⎞
               ⎜      0      Bk[r, θ]     0           0      ⎟
               ⎜      0         0      Ck[r, θ]       0      ⎟
               ⎝ -Erot[r, θ]    0         0       Dk[r, θ]  ⎠;
```

Rules for the $\mathcal{UD}$ toolkit

Instead of sparse matrices, we prefer rules for the non-zero components for further processing.

In[79]:=
```
Clear[gDD, gUU];
```

In[80]:=
```
ansatzRules = MatrixToUDRules[ansatzMetric, gDD, 𝒟, coords]
```

Out[80]=
```
{gDD t t → -Ak[r, θ], gDD t ϕ → -Erot[r, θ], gDD r r → Bk[r, θ],
 gDD θ θ → Ck[r, θ], gDD ϕ t → -Erot[r, θ], gDD ϕ ϕ → Dk[r, θ]}
```

Symbolic inverse

In[81]:=
```
ansatzInverse = Simplify[Inverse[ansatzMetric]];
ansatzInvRules = MatrixToUDRules[ansatzInverse, gUU, 𝒰, coords]
```

Out[82]=

$$\left\{gUU^{t\ t} \to -\frac{Dk[r,\theta]}{Ak[r,\theta]\times Dk[r,\theta]+Erot[r,\theta]^2},\right.$$
$$gUU^{t\ \phi} \to -\frac{Erot[r,\theta]}{Ak[r,\theta]\times Dk[r,\theta]+Erot[r,\theta]^2},\ gUU^{r\ r} \to \frac{1}{Bk[r,\theta]},\ gUU^{\theta\ \theta} \to \frac{1}{Ck[r,\theta]},$$
$$\left.gUU^{\phi\ t} \to -\frac{Erot[r,\theta]}{Ak[r,\theta]\times Dk[r,\theta]+Erot[r,\theta]^2},\ gUU^{\phi\ \phi} \to \frac{Ak[r,\theta]}{Ak[r,\theta]\times Dk[r,\theta]+Erot[r,\theta]^2}\right\}$$

Append catch-all zero rules;

In[83]:=
```
ansatzRules = Join[ansatzRules, {gDD[__] → 0}];
ansatzInvRules = Join[ansatzInvRules, {gUU[__] → 0}];
```

Ricci via the Toolkit

Toolkit function **CalculateRicciComponent** requires a Riemann-getter function. The Riemann-getter calls toolkit function **CalculateRiemannComponent**, which requires a Γ-getter function. Define that first.

Γ-getter

In[85]:=
```
ansatzGamma[s_, m_, n_] :=
  CalculateGammaComponent[s, m, n, gDD, ansatzRules, gUU, ansatzInvRules, coords];
```

Vacuum Field Equations (Ricci = 0)

In[86]:=
```
ansatzRicci[u_, v_] :=
  CalculateRicciComponent[ (*toolkit function*)
   u, v,
   Function[{a, b, c, d}, (*Riemann-getter as anon function*)
    CalculateRiemannComponent[a, b, c, d, (*toolkit function*)
     ansatzGamma, (*Γ-getter defined above*)
     coords]],
   coords];
```

It's only useful to show a smattering of these equations, so we apply **Short** to them.

In[87]:=
```
(vacuumFieldEquations = (Table[
        {a, b} → ansatzRicci[a, b] == 0,
        {a, coords}, {b, coords}] //
      Flatten //
     Association)) //
 Short[#, 8] &
```

Out[87]//Short=

$$\left\langle\left|\{t,\ t\} \to\right.\right.$$

```
(Bk[r, θ]^2 ((Ak[r, θ] × Dk[r, θ] + Erot[r, θ]^2) Ak^(0,1)[r, θ] Ck^(0,1)[r, θ] + Ck[r, θ]
          (-Ak[r, θ] (Ak^(0,1)[r, θ] Dk^(0,1)[r, θ] + 2 Erot^(0,1)[r, θ]^2) +
            Dk[r, θ] (Ak^(0,1)[r, θ]^2 - 2 Ak[r, θ] Ak^(0,2)[r, θ]) +
            2 Erot[r, θ] (Ak^(0,1)[r, θ] Erot^(0,1)[r, θ] - Erot[r, θ] Ak^(0,2)[r, θ]))) +
      <<1>> - Bk[r, θ] × Ck[r, θ] (-Ck[r, θ] × Dk[r, θ] Ak^(1,0)[r, θ]^2 -
         2 Ck[r, θ] × Erot[r, θ] Ak^(1,0)[r, θ] Erot^(1,0)[r, θ] + Erot[r, θ]^2 (Ak^(0,1)[r, θ]
             Bk^(0,1)[r, θ] + Ak^(1,0)[r, θ] Ck^(1,0)[r, θ] + 2 Ck[r, θ] Ak^(2,0)[r, θ]) +
         Ak[r, θ] (Ck[r, θ] (Ak^(1,0)[r, θ] Dk^(1,0)[r, θ] + 2 Erot^(1,0)[r, θ]^2) +
            Dk[r, θ] (Ak^(0,1)[r, θ] Bk^(0,1)[r, θ] + Ak^(1,0)[r, θ]
                Ck^(1,0)[r, θ] + 2 Ck[r, θ] Ak^(2,0)[r, θ])))) /
   (4 Bk[r, θ]^2 Ck[r, θ]^2 (Ak[r, θ] × Dk[r, θ] + Erot[r, θ]^2)) ==
  0, {t,
  r} →
 True,
{t,
  θ} →
 True,
{t,
  ϕ} →
 (Bk[r, θ]^2 ((Ak[r, θ] × Dk[r, θ] + Erot[r, θ]^2) Ck^(0,1)[r, θ] Erot^(0,1)[r, θ] +
        Ck[r, θ] (<<1>>)) + <<1>> - <<1>>) /
   (4 Bk[r, θ]^2 Ck[r, θ]^2 (Ak[r, θ] × Dk[r, θ] + Erot[r, θ]^2)) ==
  0, {r, t} → True, <<6>>, {θ,
  ϕ} →
 True,
{ϕ,
  t} →
          <<1>>
 ------------------------ ==
 4 <<2>> (<<1>> <<1>> + <<1>>)
  0, {ϕ,
  r} →
 True, {ϕ,
  θ} → True, {ϕ,
  ϕ} →
                                1
 ------------------------------------------------------------
 4 Bk[r, θ]^2 Ck[r, θ]^2 (Ak[r, θ] × Dk[r, θ] + Erot[r, θ]^2)
  (Bk[r, θ]^2 (-((Ak[r, θ] × Dk[r, θ] + Erot[r, θ]^2) Ck^(0,1)[r, θ] Dk^(0,1)[r, θ]) +
       Ck[r, θ] (-Ak[r, θ] Dk^(0,1)[r, θ]^2 - 2 Erot[r, θ] Dk^(0,1)[r, θ] Erot^(0,1)[r, θ] +
          2 Erot[r, θ]^2 Dk^(0,2)[r, θ] + Dk[r, θ]
            (Ak^(0,1)[r, θ] Dk^(0,1)[r, θ] + 2 Erot^(0,1)[r, θ]^2 + 2 Ak[r, θ] Dk^(0,2)[r, θ]))) -
```

```
<<1>>^2 <<2>> <<1>> + Bk[r, θ] × Ck[r, θ] (<<1>>)) == 0 |⟩
```

The components of the Ricci tensor are large, unwieldy expressions. Though not challenging for Wolfram to write, we did not see how to get Wolfram to solve. It's not instructive to print them out, but it is instructive to illustrate how big and unstructured they are, to make the point that we should not try to solve them directly, but find another way. We won't be using the **vacuumFieldEquations** above in the following sections, which take a different approach.

Exhibit a Component, R_{rr}

It's big

In[88]:=
```
ansatzRicci[r, r] // LeafCount
```
Out[88]=
```
753
```

In[89]:=
```
ExpressionGraph[ansatzRicci[r, r]] // EdgeCount
```
Out[89]=
```
558
```

In[90]:=
```
ExpressionGraph[ansatzRicci[r, r]] // VertexCount
```
Out[90]=
```
559
```

It's complicated: not a lot of regularity.

Optional: take the semicolon off the end of this this cell if you want to see an overwhelming, large graph, which is NOT necessary to understand the rest of the story.

In[91]:=
```
ExpressionTree[ansatzRicci[r, r], "Heads",
  TreeLayout → "RadialEmbedding", ImageSize → Full];
```

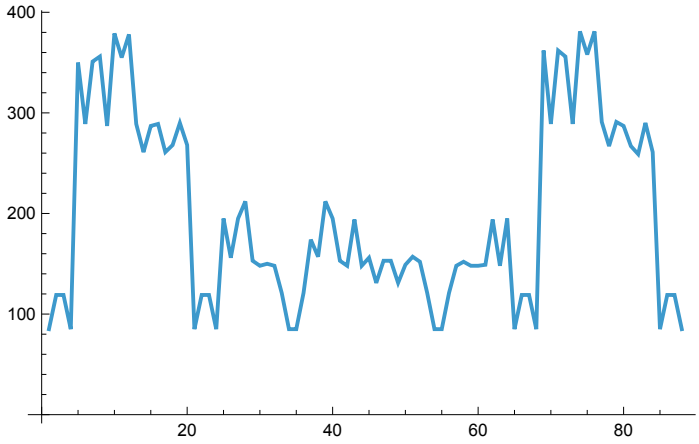
Riemann of the Ansatz

Here we see the leaf counts of the 88 non-zero components of the Kerr curvature tensor. Symmetries and antisymmetries are immediately visible.

In[92]:=
```
ansatzRiemann = Table[
    CalculateRiemannComponent[a, b, c, d, ansatzGamma, coords],
    {a, coords}, {b, coords}, {c, coords}, {d, coords}];
```

In[93]:=
```
ansatzRiemann // Flatten // Select[#, # =!= 0 &] & // Map[LeafCount, #] & // ListLinePlot
```

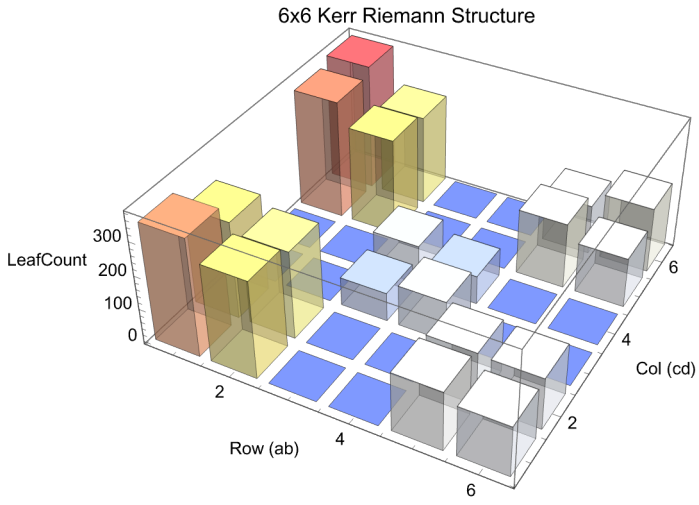
Out[93]=



Here, we see a Petrov map of leaf counts. Off-diagonal skyscrapers (R_{14}, R_{26}, etc.) represent “gravito-magnetism” or frame dragging. It says, for example: “If I walk my vector around in the {x, y} plane, it gets twisted into the {t, z} plane.”

In[94]:=
```
bivectorMap = <|{1, 2} → 1, {1, 3} → 2, {1, 4} → 3, {2, 3} → 4, {2, 4} → 5, {3, 4} → 6|>;
petrovMatrix = ConstantArray[0, {6, 6}];
Do[
  Do[
   (expr = ansatzRiemann[[i, j, k, l]];
    val = If[PossibleZeroQ[expr], 0, LeafCount[expr]];
    row = bivectorMap[Sort[{i, j}]];
    col = bivectorMap[Sort[{k, l}]];
    petrovMatrix[[row, col]] += Abs[val];),
   {k, 1, 3}, {l, k + 1, 4}],
  {i, 1, 3}, {j, i + 1, 4}];
DiscretePlot3D[petrovMatrix[[row, col]], {row, 1, 6}, {col, 1, 6},
 ExtentSize → 0.8,
 PlotRange → All,
 AxesLabel → {"Row (ab)", "Col (cd)", "LeafCount"},
 ColorFunction → "TemperatureMap",
 PlotLabel → "6x6 Kerr Riemann Structure"]
```

Out[96]=



The Wizard's Roadmap: Linearized Weak-Field Gravity

We beat a close escape from the dragon's fire to study the parchment Chandra slipped into our backpack. It is a map of the "Far Lands"—the weak-field limit beyond the dragon, where gravity is gentle and the dragon's roar is but a whisper.

Chandra's map suggests a sanity check: if our simplified ansatz is correct, then far down the road from the black hole ($r \to \infty$), the terrifying E_{rot} term should dwindle into something recognizable. It should look like a "dipole"—a gravitational magnet.

We linearize our new ansatz metric, stripping away higher-order terms. We are left with an easy equation. If we solve it, and it reveals a dipole, we know that despite the smoke and fire ahead of us, we are at least

confronting the right beast blocking the right road.

Instead of trying to solve for E_{rot} directly, check the *form* of the new ansatz metric in the weak-field approximation, far from the black hole, assuming only t—ϕ coupling off the diagonal:

Weak Metric

In[97]:=
```
Clear[h];
```

Minkowski in spherical coordinates plus a small, unknown twist, $h[r, \theta]$ in place of the full E_{rot}:

In[98]:=
```
weakMetric = {{-1, 0, 0, -h[r, θ]}, {0, 1, 0, 0}, {0, 0, r^2, 0}, {-h[r, θ], 0, 0, r^2 Sin[θ]^2}};
```

In[99]:=
```
(weakMetricInv = Simplify[Inverse[weakMetric]]) // MatrixForm
```

Out[99]//MatrixForm=

$$\begin{pmatrix} -\frac{r^2 \sin[\theta]^2}{h[r,\theta]^2+r^2 \sin[\theta]^2} & 0 & 0 & -\frac{h[r,\theta]}{h[r,\theta]^2+r^2 \sin[\theta]^2} \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{1}{r^2} & 0 \\ -\frac{h[r,\theta]}{h[r,\theta]^2+r^2 \sin[\theta]^2} & 0 & 0 & \frac{1}{h[r,\theta]^2+r^2 \sin[\theta]^2} \end{pmatrix}$$

Linearize the Metric Inverse

In[100]:=
```
weakMetricInvLinearized = weakMetricInv /. {h[r, θ]^2 → 0}
```

Out[100]=

$$\left\{\left\{-1, 0, 0, -\frac{\text{Csc}[\theta]^2\, h[r, \theta]}{r^2}\right\}, \{0, 1, 0, 0\}, \left\{0, 0, \frac{1}{r^2}, 0\right\}, \left\{-\frac{\text{Csc}[\theta]^2\, h[r, \theta]}{r^2}, 0, 0, \frac{\text{Csc}[\theta]^2}{r^2}\right\}\right\}$$

Rules for the Toolkit

As usual, we prefer rules over matrices:

In[101]:=
```
weakRules = MatrixToUDRules[weakMetric, gDD, 𝒟, coords]
```

Out[101]=

$$\{\text{gDD}_{t\,t} \to -1,\ \text{gDD}_{t\,\phi} \to -h[r, \theta],\ \text{gDD}_{r\,r} \to 1,\ \text{gDD}_{\theta\,\theta} \to r^2,\ \text{gDD}_{\phi\,t} \to -h[r, \theta],\ \text{gDD}_{\phi\,\phi} \to r^2 \text{Sin}[\theta]^2\}$$

In[102]:=
```
weakInvRules = MatrixToUDRules[weakMetricInvLinearized, gUU, 𝒰, coords]
```

Out[102]=
$$\left\{gUU^{t\ t} \to -1,\ gUU^{t\ \phi} \to -\frac{\text{Csc}[\theta]^2\, h[r,\theta]}{r^2},\ gUU^{r\ r} \to 1,\right.$$
$$\left. gUU^{\theta\ \theta} \to \frac{1}{r^2},\ gUU^{\phi\ t} \to -\frac{\text{Csc}[\theta]^2\, h[r,\theta]}{r^2},\ gUU^{\phi\ \phi} \to \frac{\text{Csc}[\theta]^2}{r^2}\right\}$$

Join with zero rules, as before:

In[103]:=
```
weakRules = Join[weakRules, {gDD[__] → 0}];
weakInvRules = Join[weakInvRules, {gUU[__] → 0}];
```

Linearized Vacuum Field Equation

Proceed as before.

Γ-Getter

In[105]:=
```
weakGamma[s_, m_, n_] :=
  CalculateGammaComponent[s, m, n, gDD, weakRules, gUU, weakInvRules, coords];
```

$R_{t\phi}$ has a differential equation for $h[r, \theta]$

In[106]:=
```
vacuumWeakEq =
 CalculateRicciComponent[t, ϕ,
   Function[{a, b, c, d}, (*Riemann Getter*)
    CalculateRiemannComponent[a, b, c, d,
     weakGamma, (*Γ-Getter*)
     coords]],
   coords] == 0
```

Out[106]=
$$-\frac{-\text{Cot}[\theta]\, h^{(0,1)}[r,\theta] + h^{(0,2)}[r,\theta] + r^2\, h^{(2,0)}[r,\theta]}{2\, r^2} == 0$$

Solution

This is much more tractable. We can guess then verify that $h[r, \theta] = \frac{1}{r}\sin^2(\theta)$ solves the vacuum, weak-field equation, in place of the full E_{rot}:

In[107]:=
```
testRule = h → Function[{r, θ}, 1/r Sin[θ]^2];
vacuumWeakEq /. testRule
```

Out[108]=
$$-\frac{-\frac{2\,\text{Cos}[\theta]^2}{r} + \frac{2\,\text{Sin}[\theta]^2}{r} + \frac{2\,\text{Cos}[\theta]^2 - 2\,\text{Sin}[\theta]^2}{r}}{2\, r^2} == 0$$

In[109]:=
```
(vacuumWeakEq /. testRule) // Simplify
```
Out[109]=
```
True
```

QED■

$h \sim \frac{1}{r}$, the metric term $g_{t\phi}$ looks like a Newtonian potential. The additional factor of $\sin^2(\theta)$ causes twist like a dipole out of the Keplerian plane due to angular momentum of the black hole. This is the "Gravito-magnetic potential."

The Magical Sword: Ernst Equation for Kerr

The map confirms we are on the right road, but the dragon, Ansatz Kerr, still blocks our passage. We cannot slay the dragon. There is nothing else to do but hack at the thicket. In there, the wizard told us, we will find the jewels to slot into the hilt of the Magical Sword, by which to enchant the dragon. But we should not hack with ordinary, dull blades. We will use the sword itself to hack the thicket!

The Magical Sword was forged in 1968 by Frederick Ernst. He knew that the thicket conceals fabulous jewels. Once inserted into the hilt of the blade and held in sunlight, will enchant the dragon. The separate metric components, the temporal A, the azimuthal D, and the twisting E_{rot} are not chaotic fragments, not brambles. They are the jewels we seek.

Ernst defined a potential, $\mathcal{E} = \varphi + i\,\Psi$, that enchants the beast at once. The five angry Einstein vacuum field equations collapse into a single, elegant equation: $\text{Re}[\mathcal{E}]\,\nabla^2\mathcal{E} = \nabla\mathcal{E} \bullet \nabla\mathcal{E}$.

Our faithful squire, the $\mathcal{UD}$ Toolkit, fetches the parts: the Blade (the Scalar Laplacian), the handguard (the Gradient), and the Hilt (the Gradient Squared) with its empty slots. We have only to assemble them and start hacking. If we can strike true, the thicket will cough up the jewels.

We follow Ernst's assembly papers closely, positing the metric, then showing it solves a physically equivalent equation. Space does not permit a full recapitulation of his papers, but references are included below.

The gist of Ernst is to notice that *A*, *D*, and E_{rot} behave like a single complex potential, combined into a single complex function $\mathcal{E}(r, \theta)$:

$$\mathcal{E} = 1 - \frac{2\,M}{r + i\,a\cos(\theta)} \tag{5}$$

where *a* is the ***specific angular momentum*** of the black hole: angular momentum *J* divided by mass *M*.

The Einstein equations reduce to a single non-linear equation, as opposed to five, coupled, non-linear equations:

$$\text{Re}[\mathcal{E}]\,\nabla^2\mathcal{E} = \nabla\mathcal{E} \bullet \nabla\mathcal{E} \tag{6}$$

We now proceed to show that the Kerr metric satisfies this equation.

Convenience Forms

In[110]:=
```
Σ = r^2 + a^2 Cos[θ]^2;
Δ = r^2 - 2 M r + a^2;
```

Metric Proposal

In[112]:=
```
kerrMetric = {{-(1 - 2 M r/Σ), 0, 0, -(2 M a r Sin[θ]^2/Σ)},
              {0, Σ/Δ, 0, 0},
              {0, 0, Σ, 0},
              {-(2 M a r Sin[θ]^2/Σ), 0, 0, (r^2 + a^2 + 2 M a^2 r Sin[θ]^2/Σ) Sin[θ]^2}};
```

Rules

In[113]:=
```
kerrRules = Join[
 MatrixToUDRules[kerrMetric, gDD, 𝒟, coords],
 {gDD[__] → 0}];
```

In[114]:=
```
kerrInverse = Simplify[Inverse[kerrMetric]];
kerrInvRules = Join[
 MatrixToUDRules[kerrInverse, gUU, 𝒰, coords],
 {gUU[__] → 0}];
```

Gradient, $\nabla\mathcal{E}$

In[116]:=
```
gradKerr[func_] :=
 GradRaised[func, gUU, kerrInvRules, coords]
```

Gradient Squared, $\nabla\mathcal{E}\bullet\nabla\mathcal{E}$

In[117]:=
```
gradKerrSquared[func_] :=
  GradSquared[func, gUU, kerrInvRules, coords];
```

Determinants

Needed for the covariant Laplacian

In[118]:=
```
detG = Simplify[Det[kerrMetric]];
sqrtDetG = Simplify[Sqrt[-detG]];
```

Covariant Laplacian

In[120]:=
```
laplacianKerr[func_] :=
  ScalarLaplacian[func, gUU, kerrInvRules, sqrtDetG, coords];
```

Ernst Potential

In[121]:=
```
ErnstPot = 1 - 2 M / (r + i a Cos[θ]);
```

In[122]:=
```
physAssumptions = {r ∈ Reals, θ ∈ Reals, M > 0, a ∈ Reals};
```

Split Re/Im Parts

In[123]:=
```
realPart = ComplexExpand[Re[ErnstPot]]
```

Out[123]=
$$1 - \frac{2\,M\,r}{r^2 + a^2\,\text{Cos}[\theta]^2}$$

Wrap Operators

In[124]:=
```
lapE = FullSimplify[laplacianKerr[ErnstPot], physAssumptions]
gradSqE = FullSimplify[gradKerrSquared[ErnstPot], physAssumptions]
```

Out[124]=
$$\frac{4\,M^2}{(r + i\,a\,\text{Cos}[\theta])^4}$$

Out[125]=
$$\frac{2\,M^2\left(a^2 + 2\,r\,(-2\,M + r) + a^2\,\text{Cos}[2\,\theta]\right)}{(r - i\,a\,\text{Cos}[\theta])\,(r + i\,a\,\text{Cos}[\theta])^5}$$

Ernst Equation: $\text{Re}[\mathcal{E}]\,\nabla^2\mathcal{E} - \nabla\mathcal{E}\bullet\nabla\mathcal{E}$

In[126]:=
```
lhs = realPart * lapE;
rhs = gradSqE;
```

In[128]:=
```
residual = lhs - rhs
```

Out[128]=
$$\frac{4\,M^2\left(1 - \frac{2\,M\,r}{r^2 + a^2\,\text{Cos}[\theta]^2}\right)}{(r + i\,a\,\text{Cos}[\theta])^4} - \frac{2\,M^2\left(a^2 + 2\,r\,(-2\,M + r) + a^2\,\text{Cos}[2\,\theta]\right)}{(r - i\,a\,\text{Cos}[\theta])\,(r + i\,a\,\text{Cos}[\theta])^5}$$

In[129]:=
```
residual = Simplify[residual]
```
Out[129]=
```
0
```

QED■

We've shone sunlight through the jewels. The dragon staggers off the road, stunned and dazed.

The Road Beyond the Dragon: Comparing to the WFR

Now that we are past the dragon, the details of the Wizard's Map become clear. We travel to the Great Library (the Wolfram Function Repository) to seek our prize. In the library, we shine sunlight through the jewels again, onto the pages of the forgotton tomes, and the user's manual and blueprints are fully revealed in language we understand.

We must be careful with dialects. The Wizard Chandra speaks of "specific angular momentum" ($a = J/M$)*, while the Library's books speak of "total angular momentum"* (J).

An image of the Magical Sword and its jewels, just as we have in hand, appears as well. We compare our sword to the image. It is a perfect match. The very jewels that enchanted the are *the secret key to the Wondrous Machine! Our quest is over, and we thought we had only begun! The Visualizer is no longer an incomprehensible artifact. The Wondrous Machine is a solved and understood instrument, ready for us to explore the Ergosphere and beyond.*

Retrieving the Kerr Metric from WFR

With parameters M and a, substituting $Ma = J$ for the angular-momentum expected by WFR:

In[130]:=
```
wfrKerr = ResourceFunction["MetricTensor"][
  {"Kerr", M, M * a}, (*Name and parameters in a list*)
  {t, r, θ, ϕ} ]
(wfrMatrix = Normal[wfrKerr["MatrixRepresentation"]] // FullSimplify) // MatrixForm
```
Out[130]=

MetricTensor[ Type: Covariant Symbol: $g_{\mu\nu}$ Dimensions: 4 Signature: Indeterminate]

Out[131]//MatrixForm=

$$\begin{pmatrix} -1+\frac{2Mr}{r^2+a^2\cos[\theta]^2} & 0 & 0 & -\frac{2aMr\sin[\theta]^2}{r^2+a^2\cos[\theta]^2} \\ 0 & \frac{r^2+a^2\cos[\theta]^2}{a^2-2Mr+r^2} & 0 & 0 \\ 0 & 0 & r^2+a^2\cos[\theta]^2 & 0 \\ -\frac{2aMr\sin[\theta]^2}{r^2+a^2\cos[\theta]^2} & 0 & 0 & \sin[\theta]^2\left(a^2+r^2+\frac{2a^2Mr\sin[\theta]^2}{r^2+a^2\cos[\theta]^2}\right) \end{pmatrix}$$

Just a new name for the metric defined above

In[132]:=
```
(myMatrix = kerrMetric) // MatrixForm
```

Out[132]//MatrixForm=

$$\begin{pmatrix} -1+\frac{2\,M\,r}{r^2+a^2\,\text{Cos}[\theta]^2} & 0 & 0 & -\frac{2\,a\,M\,r\,\text{Sin}[\theta]^2}{r^2+a^2\,\text{Cos}[\theta]^2} \\ 0 & \frac{r^2+a^2\,\text{Cos}[\theta]^2}{a^2-2\,M\,r+r^2} & 0 & 0 \\ 0 & 0 & r^2+a^2\,\text{Cos}[\theta]^2 & 0 \\ -\frac{2\,a\,M\,r\,\text{Sin}[\theta]^2}{r^2+a^2\,\text{Cos}[\theta]^2} & 0 & 0 & \text{Sin}[\theta]^2\left(a^2+r^2+\frac{2\,a^2\,M\,r\,\text{Sin}[\theta]^2}{r^2+a^2\,\text{Cos}[\theta]^2}\right) \end{pmatrix}$$

In[133]:=
```
myMatrix === wfrMatrix
```

Out[133]=
```
True
```

QED■

The Quest Achieved: Vacuum Solution for Kerr

Confronting Ansatz Kerr head-on got us burnt by dragon's fire and scarred by thorns in the thicket. Now, back at home, with the jewels inserted into a computer, we receive silent but joyous zeros.

Above, in *Ernst Equation for Kerr*, we already define rules for the Kerr metric and its inverse, the contravariant form. To review:

In[134]:=
```
kerrRules // Short
```

Out[134]//Short=

$$\left\{\text{gDD}_{t\,t}\to -1+\frac{2\,M\,r}{r^2+a^2\,\text{Cos}[\theta]^2},\ \text{gDD}_{t\,\phi}\to -\frac{2\,a\,M\,r\,\text{Sin}[\theta]^2}{r^2+a^2\,\text{Cos}[\theta]^2},\right.$$
$$\text{gDD}_{r\,r}\to \frac{r^2+a^2\,\text{Cos}[\theta]^2}{a^2-2\,M\,r+r^2},\ \text{gDD}_{\theta\,\theta}\to \text{<<1>>}+\text{<<1>>},\ \text{gDD}_{\phi\,t}\to -\frac{2\,a\,M\,r\,\text{<<1>>}^2}{r^2+a^2\,\text{<<1>>}},$$
$$\left.\text{gDD}_{\phi\,\phi}\to \text{Sin}[\theta]^2\left(a^2+r^2+\frac{2\,a^2\,M\,r\,\text{Sin}[\theta]^2}{r^2+a^2\,\text{Cos}[\theta]^2}\right),\ \text{gDD}[\_\_]\to 0\right\}$$

In[135]:=
```
kerrInvRules // Short[#, 2] & // FullSimplify
```

Out[135]//Short=

$$\left\{\text{gUU}^{t\,t}\to -1-\frac{4\,M\,r\left(a^2+r^2\right)}{\left(a^2+r\,(-2\,M+r)\right)\left(a^2+2\,r^2+a^2\,\text{Cos}[2\,\theta]\right)},\right.$$
$$\text{gUU}^{t\,\phi}\to -\frac{4\,a\,M\,r}{\left(a^2+r\,(-2\,M+r)\right)\left(a^2+2\,r^2+a^2\,\text{Cos}[2\,\theta]\right)},\ \text{gUU}^{r\,r}\to \frac{a^2-2\,M\,r+r^2}{r^2+a^2\,\text{Cos}[\theta]^2},$$
$$\text{gUU}^{\theta\,\theta}\to \frac{1}{r^2+\text{<<1>>}\ \text{<<1>>}},\ \text{gUU}^{\phi\,t}\to -\frac{4\,a\,M\,r}{\left(a^2+r\,(-2\,M+r)\right)\ (\text{<<1>>})},$$
$$\left.\text{gUU}^{\phi\,\phi}\to \frac{2\left(r\,(-2\,M+r)+a^2\,\text{Cos}[\theta]^2\right)\text{Csc}[\theta]^2}{\left(a^2+r\,(-2\,M+r)\right)\left(a^2+2\,r^2+a^2\,\text{Cos}[2\,\theta]\right)},\ \text{gUU}[\_\_]\to 0\right\}$$

Einstein's equations for the vacuum are $\text{Ricci}_{\text{Kerr}} = 0$. Check them.

Γ-Getter

In[136]:=
```
kerrGamma[s_, m_, n_] :=
  CalculateGammaComponent[s, m, n, gDD, kerrRules, gUU, kerrInvRules, coords];
```

Check a non-zero component

In[137]:=
```
kerrGamma[r, t, t] // FullSimplify
```

Out[137]=
$$-\frac{M\left(a^2+r\,(-2\,M+r)\right)\left(-r^2+a^2\,\text{Cos}[\theta]^2\right)}{\left(r^2+a^2\,\text{Cos}[\theta]^2\right)^3}$$

Riemann Helper

In[138]:=
```
kerrRiemann[u_, l_, n_, r_] :=
  CalculateRiemannComponent[u, l, n, r, kerrGamma, coords] // Simplify;
```

Check All Components

If this vanishes, then Kerr is a vacuum solution.

In[139]:=
```
Table[
   Sum[kerrRiemann[c, m, c, n], {c, coords}],
   {m, coords}, {n, coords}] // FullSimplify // MatrixForm
```

Out[139]//MatrixForm=
$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

QED■

Epilogue

We began this quest with a mysterious artifact : a Visualizer that produced beautiful, chaotic orbits but whose inner workings were a riddle .

The Wizard Chandra warned us that the machinery—the Kerr metric—was too complex to understand by force. He was right. Confronting the dragon (the *ab-initio* vacuum equations) earned us firethorns of algebra. But, we retreated only from the blaze, not from the quest. Consulting the Wizard' s Map (the linearized limit), we were sure that the keys lay ahead. We assembled the Magical Sword (the Ernst Equation) from provisions carried by our faithful squire, the $\mathcal{UD}$ Toolkit. With that sword, we pierced the thicket and found the enchanting jewels to pacify the dragon. The terrifying complexity of the

spinning black hole—the frame-dragging, the time dilation, the time-azimuth coupling—collapses into a single, elegant identity The Rip Tide of the Ergosphere is not a glitch, it is twisting spacetime. We showed that our faithful $\mathcal{UD}$ squire is strong enough and that our tools are sharp enough for whatever monsters lie ahead.

References

1. Ernst, F. J., *New Formulation of the Axially Symmetric Gravitational Field Problem*. Physical Review, Vol. 167, No. 5, pp. 1175-1178 (1968). [DOI: 10.1103/physrev.167.1175]
2. Ernst, F. J., *New Formulation of the Axially Symmetric Gravitational Field Problem. II.* Physical Review, Vol. 168, No. 5, pp. 1415-1417 (1968). [DOI: 10.1103/physrev.168.1415]
3. Ernst, F. J., *New Formulation of the Axially Symmetric Gravitational Field Problem. II [Erratum].* Physical Review, Vol. 172, No. 5, p. 1850 (1968). [DOI: 10.1103/physrev.172.1850.3]
4. Kerr, R. P., *Gravitational Field of a Spinning Mass as an Example of Algebraically Special Metrics.* Physical Review Letters, Vol. 11, No. 5, pp. 237-238 (1963). [DOI: 10.1103/PhysRevLett.11.237]
5. Boyer, R. H., *Maximal Analytic Extension of the Kerr Metric*. Journal of Mathematical Physics, Vol. 8, No. 2, pp. 265-281 (1967). [DOI: 10.1063/1.1705193]
6. Newman, E. T., Couch, E., Chinnapared, K., Exton, A., Prakash, A., & Torrence, R., *Metric of a Rotating, Charged Mass*. Journal of Mathematical Physics, Vol. 6, No. 6, pp. 918-919 (1965). [DOI: 10.1063/1.1704351]
7. Hartle, J. B., *Gravity: An Introduction to Einstein's General Relativity*. Benjamin Cummings (2003)."
8. Chandrasekhar, S., *The Mathematical Theory of Black Holes*. Oxford University Press (1983). Chapter 6, Section 53.
9. Beckman, Brian. *From the steam age to quantum fields: a unified approach via UD tensorial calculus*. Wolfram Community post.
10. Beckman, Brian. *General relativity in the UD calculus*. Wolfram Community post.
11. Beckman, Brian. *Formal Differential Geometry in the UD Calculus: Part 1*. Wolfram Community post.
12. Beckman, Brian. *Covariant Derivatives and Connections in the $\mathcal{UD}$ Calculus*. Wolfram Community post.
13. Beckman, Brian. *Riemann Curvature in $\mathcal{UD}$ + Compiler POC*. Wolfram Community post (to appear).
14. Beckman, Brian. *Schwarzschild Orbits in the $\mathcal{UD}$ Calculus*. Wolfram Community post (to appear).
15. Beckman, Brian. *Schwarzschild Metric from Scratch via $\mathcal{UD}$ Compilation*. Wolfram Community post (to appear).
16. Wolfram Function Repository. *MetricTensor*. For ground truth.
17. Wolfram Function Repository. *ChristoffelSymbol*. For ground truth.
18. [MTW] Misner, Charles W.; Thorne, Kip S.; Wheeler, John Archibald. *Gravitation*. Princeton University Press, 2017.

19. Petrov, A. Z. *Classification of spaces defining gravitational fields*, Scientific Transactions of Kazan State University, 114, 55 (1954)